



ПРИКЛАДНІ ШІ-АГЕНТИ ТА СИСТЕМИ НА ОСНОВІ ВЕЛИКИХ МОВНИХ МОДЕЛЕЙ

Робоча програма навчальної дисципліни (Силабус)

Реквізити навчальної дисципліни

Рівень вищої освіти	Другий (магістерський)
Галузь знань	12 Інформаційні технології
Спеціальність	122 «Комп'ютерні науки»
Освітня програма	«Системи і методи штучного інтелекту»
Статус дисципліни	Нормативна
Форма навчання	Очна (денна)
Рік підготовки, семестр	1 рік магістратури, осінній семестр
Обсяг дисципліни	120 годин / 4 кредити ЄКТС (лекції - 36 год., лабораторні роботи - 18 год., СРС - 66 год.)
Семестровий контроль / контрольні заходи	Залік / МКР
Розклад занять	http://rozklad.kpi.ua/
Мова викладання	Українська
Інформація про керівника курсу / викладачів	Лектор: PhD, Рязановський Кирило Денисович (riazanovskyi.k@kpi.ua) Практичні/Лабораторні: PhD, Рязановський Кирило Денисович (riazanovskyi.k@kpi.ua)
Розміщення курсу	Платформа дистанційного навчання (Moodle)

Програма навчальної дисципліни

1. Опис навчальної дисципліни, її мета, предмет вивчення та результати навчання

Силабус навчальної дисципліни складено відповідно до освітньої програми підготовки магістрів спеціальності 122 «Комп'ютерні науки».

Метою навчальної дисципліни є формування у студентів комплексу теоретичних знань та практичних навичок проєктування, розробки, оцінювання та розгортання прикладних систем на основі великих мовних моделей: RAG-пайплайнів, автономних ШІ-агентів, графових агентних workflow, систем виклику зовнішніх інструментів, пам'яті, моніторингу, безпеки та human-in-the-loop контролю. Основний фокус дисципліни - не вивчення окремих бібліотек, а засвоєння архітектурних принципів побудови надійних LLM-систем, які можуть бути реалізовані з використанням сучасних фреймворків і API.

Компетентності, на формування яких спрямована дисципліна:

(ЗК 1) Здатність до абстрактного мислення, аналізу, проєктування та оптимізації інтелектуальних програмних комплексів з урахуванням вимог надійності, безпеки та пояснюваності.

(ФК 1) Здатність інтегрувати великі мовні моделі у прикладні інформаційні системи за допомогою API, структурованого виводу, схем валідації та контролю якості відповідей.

(ФК 2) Здатність проєктувати архітектуру RAG-систем та автономних ШІ-агентів, включаючи модулі пам'яті, планування, виклику інструментів, трасування і human approval.

(ФК 3) Здатність створювати та оцінювати мультиагентні оркестрації для автоматизації складних бізнес- і дослідницьких процесів з урахуванням ризиків prompt injection, excessive agency та витоку даних.

Програмні результати навчання:

- (ПРН 1) Застосовувати Python та спеціалізовані фреймворки (LangChain, LangGraph, LlamaIndex, CrewAI, OpenAI Agents SDK або їх аналоги) для розробки прикладних LLM-систем.
- (ПРН 2) Розробляти та оптимізувати RAG-системи з використанням векторних баз даних, hybrid retrieval, reranking, метрик якості пошуку та аналізу hallucination/faithfulness.
- (ПРН 3) Проектувати автономних агентів із контрольованим доступом до інструментів, станом, пам'яттю, логуванням, guardrails та тестовими сценаріями для оцінки якості.
- (ПРН 4) Розгортати локальні та відкриті мовні моделі, застосовувати квантизацію, вимірювати latency, throughput, cost та якість відповіді.

2. Пререквізити та постреквізити дисципліни

Для успішного вивчення курсу студенти мають володіти знаннями з дисциплін «Програмування на Python», «Архітектура нейронних мереж», «Обробка природної мови (NLP)», а також базовими навичками роботи з API, Git та середовищами розробки. Отримані знання використовуються при виконанні магістерських робіт, розробці SaaS-рішень, інтелектуальних сервісів аналітики даних, knowledge management систем та внутрішніх AI-інструментів організацій.

3. Зміст навчальної дисципліни

Розділ 1. Інженерія LLM-систем, архітектура RAG та локальні мовні моделі

Тема 1.1. Велика мовна модель як компонент системи. Архітектура Transformer і механізм attention; шлях від тексту до відповіді: токенизація, embeddings, блоки Transformer, логіти, вибір токена. Три стадії навчання (pre-training, instruction tuning, вирівнювання через RLHF і DPO) та алгоритми декодування. Ймовірісна природа генерації, джерела недетермінізму, KV-cache і вартість контексту.

Тема 1.2. Промпт-інженерія та керування контекстом. Складові промпта, in-context learning, few-shot, chain-of-thought і self-consistency з умовами вимірювання. Context engineering: бюджет уваги, lost-in-the-middle, підвантаження за потреби, ущільнення. Межа довіри між інструкцією і даними, непрямий prompt injection.

Тема 1.3. Робота з LLM API і structured outputs. Контракт виводу через JSON Schema і Pydantic, strict-режим, обмежене декодування. Контекстне вікно, streaming, чотири класи помилок, політика повтору з відкатом, ідемпотентність, резервна модель, контроль витрат і latency.

Тема 1.4. Векторні представлення та семантичний пошук. Розмічений набір запитів і метрики Recall@k, MRR, nDCG. Зворотний індекс, idf і BM25; щільний пошук і косинусна подібність. Індеси HNSW та IVF-PQ, гібридний пошук і Reciprocal Rank Fusion; FAISS, Chroma та аналоги.

Тема 1.5. Архітектура RAG. Дві пам'яті системи, RAG проти донавчання. Ingestion: парсинг, метадані чанка, стратегії чанкінгу, contextual retrieval. Складання промпта, citation grounding як контракт коду, відмова за відсутності підстав, права доступу до пошуку, оновлення й вилучення документів.

Тема 1.6. Покращений RAG і вимірювання якості. Переписування та розкладання запиту, HyDE, sentence-window retrieval, перанкування крос-енкодером. Оцінювання retrieval quality і faithfulness, довірчі інтервали та потрібний розмір набору, ablation як спосіб довести, що прийом працює.

Тема 1.7. Локальні та відкриті мовні моделі. Сімейства Llama, Mistral, Qwen, Gemma, gpt-oss та ліцензійні режими. Фази prefill і decode, арифметика пам'яті, квантизація і її ціна в якості. Сервінг через Ollama та vLLM, конфігурація KV-кешу, патерни розгортання і межі власного заліза.

Тема 1.8. Production-архітектура LLM-сервісу. TTFT, TPOT, throughput і goodput; черги та поведінка системи під навантаженням. Кешування промпта і семантичний кеш із його ризиками, ліміти провайдера, таймаути й повтори, деградація замість відмови, observability, робота з секретами та приватність.

Розділ 2. Автономні ШІ-агенти, tool use та мультиагентні системи

Тема 2.1. Концепція автономного ШІ-агента. Цикл агента: сприйняття, стан, пам'ять, планування, інструменти, виконання та перевірка. Різниця між воркфлоу та агентом і критерій вибору. Бюджети кроків, часу, грошей і дій у коді, детектор зациклення, рівні автономії й таксономія відмов.

Тема 2.2. Tool use, function calling та протоколи інтеграції. Інструмент як типізований контракт, опис для читача-моделі, режими tool_choice, типізовані помилки виконання. Патерн ReAct і умови його виграшу. Model Context Protocol і A2A: задача M×N, примітиви, транспорти, права доступу, sandboxing та журнал аудиту.

Тема 2.3. Архітектура пам'яті агента. Робоче вікно, довготривала пам'ять і індекс знань; епізодична, семантична та процедурна пам'ять. Політики запису, оновлення й забування, отруєння пам'яті. Довготривалі процеси: контрольні точки, відновлення після збою та ідемпотентність побічних ефектів.

Тема 2.4. Фреймворки побудови агентів: LangChain і LangGraph, LlamaIndex, CrewAI, OpenAI Agents SDK. Архітектурні патерни, незалежні від конкретної бібліотеки; критерії вибору за підтримкою стану, контрольних точок і людини в контурі.

Тема 2.5. Графові агенти та керування станом. Явний стан і редьюсери, умовна маршрутизація, цикли й супер-кроки. Контрольні точки та відновлення сеансу, історія станів і повернення назад, human-in-the-loop як властивість топології графа, а не елемента інтерфейсу.

Тема 2.6. Мультиагентні системи: supervisor, ієрархія, мережа, handoff. Ізоляція контексту як справжня причина виграшу, координаційна вартість і токен-бюджет. Виміряні прирости та їх межі, режими відмов і атрибуція винуватця, чекліст допуску архітектури.

Тема 2.7. Оцінювання, observability та безпека агентних систем. Golden set, рівні оцінювання, task success і tool-call accuracy, pass@k проти pass@k, LLM-суддя та його калібрування. Трасування й семантичні конвенції OpenTelemetry GenAI. Prompt injection і смертельна триада, OWASP Top 10 для LLM-застосунків і для агентних систем, guardrails, tool misuse, excessive agency, архітектурні контрзаходи та evidence package.

4. Навчальні матеріали та ресурси

Базова література:

1. Jurafsky, D., Martin, J. H. (2026). *Speech and Language Processing*. 3rd edition draft, January 2026. Розділи 2, 5, 7, 8, 10, 11. Вільний доступ: web.stanford.edu/~jurafsky/slp3/
2. Xiao, T., Zhu, J. (2025). *Foundations of Large Language Models*. arXiv:2501.09223. CC BY-NC 4.0. Розділи 1–5.
3. Raschka, S. (2024). *Build a Large Language Model (From Scratch)*. Manning Publications. Код і рисунки: github.com/rasbt/LLMs-from-scratch
4. Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS. arXiv:1706.03762.
5. Anthropic (2024). *Building Effective Agents*; Anthropic (2025). *Effective Context Engineering for AI Agents*. Інженерні публікації.
6. Офіційна документація: Model Context Protocol, LangGraph, OpenAI Agents SDK, vLLM, Ollama, Hugging Face Transformers. Версії перевіряються на початку семестру.

Допоміжна література, статті та стандарти:

7. Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR. arXiv:2210.03629.
8. Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS. arXiv:2005.11401.
9. Gao, Y. et al. (2024). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997.
10. Liu, N. F. et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts*. TACL. arXiv:2307.03172.
11. Kwon, W. et al. (2023). *Efficient Memory Management for LLM Serving with PagedAttention*. SOSP. arXiv:2309.06180.
12. Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS. arXiv:2306.05685.
13. Miller, E. (2024). *Adding Error Bars to Evals*. Anthropic. arXiv:2411.00640.
14. Cemri, M. et al. (2025). *Why Do Multi-Agent LLM Systems Fail?* arXiv:2503.13657.
15. Rafailov, R. et al. (2023). *Direct Preference Optimization*. NeurIPS. arXiv:2305.18290.
16. Greshake, K. et al. (2023). *Not What You've Signed Up For: Indirect Prompt Injection*. AISec. arXiv:2302.12173.
17. DeBenedetti, E. et al. (2024). *AgentDojo*. NeurIPS. arXiv:2406.13352; Nasr, M. et al. (2025). *The Attacker Moves Second*. arXiv:2510.09023.
18. OWASP GenAI Security Project. OWASP Top 10 for LLM Applications та Top 10 for Agentic Applications (актуальні редакції); NIST AI 600-1. *Generative AI Profile*.

Навчальний контент

5. Методика опанування навчальної дисципліни

Лекційні заняття (36 годин / 18 пар)

№	Назва теми лекції та перелік основних питань
1	Тема 1.1. Велика мовна модель: архітектура

	Transformer і поведінка в системі. Топологія моделі: текст, токени, embeddings, блоки Transformer, логіти, вибір токена. Механізм attention. Ймовірна природа генерації та п'ять джерел недетермінізму. Дев'ять компонентів LLM-системи, критерії вибору моделі, чому оцінювання проєктують першим.
2	Тема 1.1. Токени, embeddings і attention: що модель бачить на вході. Токенізація і BPE, розмір словника, токен-бюджет і його залежність від мови. Векторні представлення, косинусна подібність і межі її застосування. Scaled dot-product attention, маскування, позиційне кодування, квадратична вартість контексту, KV-cache і згрупована увага.
3	Тема 1.1. Як навчають модель: pre-training, вирівнювання та декодування. Три стадії навчання й одна стадія виконання. Функція втрат передтренування, закони масштабування та їхні межі. Instruction tuning, RLHF з KL-штрафом, DPO. Алгоритми декодування: greedy, температура, top-k, top-p; чому нульова температура не гарантує відтворюваності.
4	Тема 1.2. Промпт-інженерія та керування контекстом. Шість складових промпта, in-context learning, few-shot, chain-of-thought і self-consistency з умовами вимірювання; спад ефекту CoT на reasoning-моделях. Context engineering: бюджет уваги, lost-in-the-middle, підвантаження за потреби, ущільнення, нотатки поза вікном. Межа довіри між інструкцією і даними, непрямий prompt injection.
5	Тема 1.3. Structured outputs: як зробити відповідь моделі придатною для коду. Контракт виводу як схема, перевірка й визначена поведінка при порушенні. JSON Schema і Pydantic, strict-режим та його підмножина, обмежене декодування. Чотири класи помилок, політика повтору, ідемпотентність, відкат із джитером, circuit breaker і резервна модель, облік токенів і вартості.
6	Тема 1.4. Векторний пошук: як знайти потрібний фрагмент серед мільйона. Постановка задачі через фрагменти й розмічений набір запитів. Recall@k, MRR, nDCG. Зворотний індекс, idf і BM25 з параметрами k_1 та b . Щільний пошук і косинус, один простір на один індекс. HNSW і його ручні, IVF-PQ, гібридний пошук і Reciprocal Rank Fusion.
7	Тема 1.5. Архітектура RAG: від документа до відповіді з цитатою. Дві пам'яті системи, формула маргіналізації, RAG проти донавчання. Ingestion: парсинг, метадані чанка й операції, які вони виконують. Стратегії чанкінгу, contextual retrieval. Складання промпта, citation grounding як контракт коду, відмова при порожньому пошуку, локалізація помилки за симптомом.
8	Тема 1.6. Якість RAG: переписування запиту,

	перанкування та вимірювання. Розділення метрик пошуку і генерації. Розмір набору й довірчий інтервал: яку різницю здатен побачити ваш набір. Переписування запиту, multi-query, HyDE та їхня ціна. Кросенкодер і двоетапна схема. Faithfulness через розкладання на твердження, межі автоматичного оцінювання, ablation зі зміною однієї речі.
9	Тема 1.7. Локальні та відкриті моделі: пам'ять, квантизація, сервінг. Своє чи чуже залізо: критерій за критерієм. Prefill і decode, чому декод упирається в пропускну здатність пам'яті. Бюджет пам'яті картки й кількість одночасних сеансів. Квантизація GPTQ, AWQ, GGUF, FP8 і те, що ламається першим. PagedAttention, неперервне батчування, кешування префіксів. Сімейства відкритих ваг і їхні ліцензії.
10	Тема 1.8. Production-архітектура LLM-сервісу: потужність, кеші, деградація. TTFT, TPOT, throughput і goodput. Планування потужності від інтенсивності запитів до кількості карток, закон Літтла, чому цільове завантаження 0,7. Три різні кеші та ризик кожного. Бюджет часу по етапах, маршрутизація за складністю, ліміти провайдера, черга з пріоритетами, таблиця деградації, вартість на успішну задачу, SLO.
11	Тема 2.1. Агент як керований цикл: стан, бюджети, межа автономії. Означення агента і workflow: хто обирає наступний крок. П'ять патернів із фіксованим маршрутом. Сім складових кроку циклу, критерій зупинки. Чотири бюджети запуску, білий список дій і зворотність, рівні автономії. Таксономія відмов агентних систем і три найчастіші режими.
12	Тема 2.2. Tool use і ReAct: як модель просить систему діяти. Шість полів оголошення інструмента, опис як частина промпта, три діалекти схем. П'ять кроків обміну, режими tool_choice, паралельні виклики та межа набору. ReAct: розширений простір дій і його виміряні результати. Типізовані помилки, з яких впливає наступна дія. Найменші привілеї, пісочниця, аудит-лог.
13	Тема 2.2. MCP і A2A: спільний протокол замість власних конекторів. Задача M×N проти M+N. Три ролі та три примітиви MCP, транспорти stdio і Streamable HTTP, форма повідомлення JSON-RPC, версіювання ревізій і stateless-модель. A2A: Agent Card, життєвий цикл задачі, термінальні стани. Протокол як не-межа довіри: ризики інтеграції та залишковий ризик.
14	Тема 2.3. Пам'ять агента і довготривалі процеси. Три сховища з різними термінами життя та правом запису. Таксономія типів пам'яті й межі її формалізації. Архітектури: пейджинг між вікном і сховищем, потік записів з оцінкою добору,

	часовий граф фактів. Політики запису й забування, отруєння пам'яті. Довговічне виконання, checkpoint, пауза на людину і пастка подвійного побічного ефекту.
15	Тема 2.4, 2.5. Графові агенти: явний стан, checkpoint і людина в контурі. Чому неявний цикл не можна ні відновити, ні зупинити. Вузол, ребро, стан із редьюсером, супер-крок. Цикли й лічильники в стані, підграфи. Checkpointer, відновлення, повтор і розгалуження. Три типи втручання людини й одна механіка зупинки. Порівняння LangGraph, OpenAI Agents SDK, CrewAI, AutoGen, Llamaindex Workflows.
16	Тема 2.6. Мультиагентні системи: коли другий агент окупається. Що множить другий агент: контексти, права, критерії зупинки. Топології та кількість каналів обміну. Опубліковані прирости й умови їх вимірювання; реплікації, які частину з них скасували. Координаційна вартість. Режими відмов і атрибуція винуватця. Чекліст із семи питань до допуску архітектури, механіка handoff.
17	Тема 2.7. Оцінювання та observability agentic-систем. Golden set і вимоги до нього. П'ять рівнів оцінювання. Task success, tool-call accuracy, groundedness; pass ^k проти pass@k і чому середня точність не є надійністю. Вартість як метрика якості. LLM-суддя: три режими, виміряні упередження, калібрування та каппа. Довірчі інтервали й розмір набору. Трасування, семантичні конвенції OpenTelemetry GenAI, замикання циклу.
18	Тема 2.7. Безпека агентів і доказ готовності системи. Чому недовірений текст стає інструкцією. Смертельна тріада і правило двох із трьох. OWASP Top 10 для LLM-застосунків та окремий перелік для агентних. Канали доставки ін'єкції, задокументовані інциденти. Виміряна ефективність захистів і те, що встояло під адаптивною атакою. Архітектурні контрзаходи, приватність і регуляторні вимоги, evidence package.

Лабораторні заняття (18 годин / 9 пар)

Навчально-методичне забезпечення: комплект з 18 лекційних презентацій (25-44 слайди) з визначеннями за першоджерелами, схемами архітектур із офіційної документації та наукових статей, робочими прикладами коду на Python і клікабельними посиланнями на джерела; наскрізний приклад системи, що розвивається від typed-клієнта до захищеного агента; методичні вказівки до п'яти лабораторних робіт, що виконуються бригадами по три особи, з розподілом ролей, порядком виконання, вимогами до захисту та контрольними питаннями; нотатки лектора з підготовкою до кожного заняття, переліком спрощень та ілюстративних чисел; звіт перевірки тверджень за джерелами. Матеріали розміщено у Moodle.

№	Назва теми занять та перелік основних питань	Години
1	Лабораторна робота 1. Локальний сервінг і надійний typed-клієнт. Бригада з 3 осіб:	3

	сервінг і вимірювання (TTFT, TPOT, бюджет пам'яті двох моделей); контракт виводу через JSON Schema і Pydantic, обмежене декодування, чотири класи помилок; надійність — ретраї з джитером, таймаути, запобіжник, ідемпотентність, 40+ випадків і тести. Інтеграція: HTTP-сервіс поверх обох моделей і навантажувальний тест. Усе локально, без платних API. Здається один Jupyter-ноутбук на бригаду: код учасників із коментарями, JSON Schema контракту, таблиця порівняння двох моделей, таблиця класів помилок, висновок. Лекції 1–5, 9.	
2	Лабораторна робота 2. RAG-система над відкритим корпусом. Бригада з 3 осіб: корпус із ліцензіями та ingestion з метаданими; розріджений і щільний індекси, гібрид через RRF, кросенкодерний реранкер; розмічений набір із 60+ запитів, метрики та довірчі інтервали парних різниць Recall@5, генерація з цитуванням і відмовою. Спільний ablation і демонстрація. Здається Jupyter-ноутбук: таблиця джерел і ліцензій, чотири конфігурації з інтервалами парних різниць, ablation, приклади відповідей із цитатами. Лекції 6–8.	4
3	Лабораторна робота 3. Агент з інструментами, пам'яттю і людиною в контурі. Бригада з 3 осіб: чотири інструменти з JSON-схемами, типізовані помилки шести класів і таблиця прав; граф станів LangGraph, checkpointер із durability=sync, три сховища пам'яті, переривання перед незворотною дією; набір із 25 задач, три прогони, класифікація відмов за таксономією. Демонстрація падіння і відновлення без подвоєння запису. Здається Jupyter-ноутбук: схема графа, лог відновлення, таблиця прогонів, таблиця відмов за класами. Лекції 11, 12, 14, 15.	4

4	Лабораторна робота 4. MCP-сервер, мультиагентна конфігурація та доказ користі. Бригада з 3 осіб: MCP-сервер лише для читання з тестами обох рівнів помилок і аудит-логом, підключення до робочого агента; одиночний агент і supervisor з двома виконавцями на однакових інструментах; набір із 40 задач, $pass^3$, довірчий інтервал парної різниці, калібрування LLM-судді через каппу, траси за конвенціями OpenTelemetry. Здається Jupyter-ноутбук: тести обох рівнів помилок, порівняльна таблиця з інтервалом різниці, вартість координат, явне рішення про архітектуру. Лекції 13, 16, 17.	4
5	Лабораторна робота 5. Автоматизація процесу в n8n і безпека агента. Бригада з 3 осіб: self-hosted n8n у Docker, воркфлоу з тригером, гілками і обробкою помилок, локальна модель вузлом Ollama; агентний вузол з інструментами й MCP-сервером, підтвердження людини перед незворотним кроком, схема меж довіри; 20 сценаріїв prompt injection із машинно перевірюваними критеріями, вимірювання кожного захисту окремо. Усе безкоштовне: n8n self-hosted і локальна модель. Здається Jupyter-ноутбук разом з експортом воркфлоу і схемою меж довіри: таблиця атак до і після кожного захисту, висновок. Лекції 10, 13, 18.	3

6. Самостійна робота студента

№ з/п	Вид самостійної роботи	Кількість годин СРС
1	Опрацювання лекційного матеріалу, першоджерел і документації	18
2	Реалізація typed-клієнта, тести і звіт до Лаб. №1	6
3	Збирання корпусу, реалізація RAG-пайплайну, ablation і звіт до Лаб. №2	8
4	Реалізація агента, графа станів, checkpoint і звіт до Лаб. №3	8
5	Реалізація MCP-сервера,	8

	мультиагентного варіанта, eval-набору і звіт до Лаб. №4	
6	Розгортання локальної моделі, навантажувальні та безпекові тести, звіт до Лаб. №5	6
7	Підготовка до модульної контрольної роботи (МКР)	6
8	Підготовка до семестрового контролю (заліку)	6
	Всього:	66

7. Контрольні роботи

Модульна контрольна робота проводиться наприкінці семестру у вигляді комп'ютерного тестування (Moodle) та практичного відкритого завдання. Практичне завдання фокусується на проєктуванні архітектури ШІ-агента або RAG-системи для конкретного сценарію: визначення станів, інструментів, політик доступу, метрик оцінювання, тестового набору, можливих failure modes та захисних бар'єрів. Усі 18 лекційних пар відведено під навчальний матеріал, тому теоретичну частину МКР студенти складають у Moodle у визначений викладачем тиждень поза лекційним часом, а відкрите практичне завдання захищають на останньому лабораторному занятті.

Політика та контроль

8. Політика навчальної дисципліни

Правила відвідування занять: присутність на лекціях не оцінюється балами, однак через високу швидкість розвитку Generative AI та складність побудови agentic workflows відвідування занять є важливим для своєчасного виконання лабораторних робіт.

Політика щодо академічної доброчесності: лабораторні роботи виконуються самостійно. Використання LLM-асистентів (GitHub Copilot, ChatGPT тощо) дозволяється як інструмент розробки, але студент зобов'язаний зазначити факт використання, надати власні тести та повністю пояснити архітектуру, код, промпти, метрики і обмеження рішення під час захисту. Сліпе копіювання згенерованого коду або чужих робіт без розуміння логіки не зараховується.

Політика дедлайнів: захист кожної лабораторної роботи має відбутися у встановлений викладачем термін. Роботи, здані після дедлайну без поважної причини, оцінюються зі зниженням максимального балу.

Безпека та робота з даними: у лабораторних роботах забороняється використовувати реальні конфіденційні дані без анонімізації. Інструменти агента мають працювати з мінімально необхідними дозволами; потенційно ризикові дії мають вимагати підтвердження людини.

9. Види контролю та рейтингова система оцінювання (PCO)

Максимальний рейтинг студента складає 100 балів. Стартовий семестровий рейтинг становить 80 балів: 50 балів за лабораторні роботи та 30 балів за модульну контрольну роботу. Залікова контрольна робота оцінюється у 20 балів.

Метод оцінювання	Кількість	Мінімальний бал (допуск)	Максимальний бал	Всього балів
Виконання та захист лабораторних робіт	5	5 за кожную	10 за кожную	50
Модульна контрольна робота (МКР)	1	0	30	30
Максимальний стартовий рейтинг				80
Залікова контрольна робота	1	0	20	20
Загальний підсумковий рейтинг				100

Критерії оцінювання лабораторних робіт:

9-10 балів: завдання виконано повністю, код працює стабільно, рішення має зрозумілу архітектуру, звіт містить метрики та аналіз помилок, студент вільно захищає роботу.

7-8 балів: завдання виконано, але є незначні технічні або методичні зауваження, неповний аналіз метрик чи невеликі порушення строків здачі.

5-6 балів: завдання виконано частково, результати нестабільні або захист демонструє недостатнє розуміння використаних інструментів.

0 балів: програма не працює, робота не захищена або виявлено критичний плагіат / сліпе копіювання без розуміння.

МКР оцінюється у 30 балів: теоретична частина у Moodle - до 15 балів; відкрите практичне завдання на проектування системи та аналіз ризиків - до 15 балів.

Умови допуску до заліку: успішний захист усіх 5 лабораторних робіт та стартовий рейтинг не менше 36 балів.

Студенти, які набрали стартовий рейтинг 60 балів і більше та виконали умови допуску, можуть отримати залік автоматично. Для автоматичного заліку підсумкова оцінка визначається шляхом масштабування стартового рейтингу до 100-бальної шкали: $R = \text{round}(R_{\text{ст}} \times 100 / 80)$. Студенти, які бажають підвищити результат або мають стартовий рейтинг менше 60 балів, складають залікову контрольну роботу; у цьому випадку підсумковий рейтинг обчислюється як $R = R_{\text{ст}} + R_{\text{залік}}$, але не більше 100 балів.

Кількість балів	Оцінка
95 - 100	Відмінно
85 - 94	Дуже добре
75 - 84	Добре
65 - 74	Задовільно
60 - 64	Достатньо
Менше 60	Незадовільно
Менше 36 або не всі лабораторні роботи	Не допущено

10. Додаткова інформація з дисципліни

Студентам може бути зараховано додаткові (бонусні) бали до стартового рейтингу за проходження профільних курсів DeepLearning.AI, LangChain, Coursera або аналогічних програм за напрямками AI Agents, RAG, LLMOps чи LangChain/LangGraph for LLM Application Development за умови надання сертифіката та короткої демонстрації отриманих навичок. Загальна сума балів не може перевищувати 100.

Робочу програму навчальної дисципліни (силабус):

Складено: PhD, Рязановський Кирило Денисович

Ухвалено кафедрою штучного інтелекту (протокол № __ від __.__.202__)

Погоджено Методичною комісією факультету (протокол № __ від __.__.202__)